

Software Engineering of Multi-Sensor Counter-UAS Decision-Support Platforms: An Event-Driven Command-and-Control Monograph

Author: Victor.I

Programme: Counter-Swarm Defence — Research Monographs (Software Engineering)

Scope: Defensive decision-support software only. No weapon control, kinetic fire-control, electronic-attack execution, or effector guidance.

Companion artefacts: `architecture/system.md`, `architecture/data-flow.md`, `architecture/interfaces.md`, `architecture/failure-modes.md`, `docs/systems/testing-strategy.md`, `docs/systems/observability.md`, `docs/software/architecture-notes.md`

Abstract

Multi-sensor counter-unmanned aircraft system (counter-UAS) platforms must convert heterogeneous, late, duplicated, and sometimes contradictory observations into an operator-usable air picture and graded risk support under strict safety and audit constraints. The software-engineering problem is not primarily algorithmic novelty; it is the long-horizon design of a reliable, observable, contract-driven distributed system whose failure modes remain visible and whose outputs never silently cross from recommendation into actuation.

This monograph develops a structured research treatment of that problem for defensive decision-support platforms, using the Counter-Swarm Defence service catalogue as the running implementation blueprint. It compares monolithic, modular-service, and event-driven styles against quality attributes drawn from ISO/IEC 25010 and operational non-functional requirements; justifies an event-streaming core with Kafka-compatible semantics as articulated by Kreps and collaborators and elaborated in contemporary Apache Kafka documentation; and examines API design, consistency, observability, testing, correctness arguments, and operational failure modes. Architectural arguments draw carefully on Newman's *Building Microservices*, Fowler's published treatments of microservices and enterprise application patterns, Kleppmann's *Designing Data-Intensive Applications*, Hohpe and Woolf's *Enterprise Integration Patterns*, Bass, Clements and Kazman's *Software Architecture in Practice*, and

related landmark works. Human-facing latency and cognitive load are treated as first-class quality attributes, with light reference to Nielsen's usability heuristics where interface contracts affect operator trust.

The selected style is a **modular event-driven core** with query-side services for the operator console; adapters and a **normaliser** produce canonical observations; **tracking** and **fusion** maintain stateful estimates; **risk** emits graded assessments with missing-information signals; and **operator-api** (with a realtime gateway) serves human decision capture without embedding effector control. Correctness is framed as a portfolio of contracts, idempotent handlers, simulation golden runs, chaos experiments mapped to FMEA identifiers, and safety interlock tests that prove forbidden side effects cannot be invoked through platform APIs. The monograph closes with an incremental build guide aligned to Counter-Swarm staging and an explicit statement of residual risks that three-year ownership must accept or retire.

Keywords: event-driven architecture; counter-UAS decision support; Kafka; microservice modularity; data consistency; observability; contract testing; chaos engineering; software architecture; defensive C2 software.

1. Introduction

1.1 Problem restatement

A defensive multi-sensor counter-UAS decision-support platform must:

1. Ingest observations from heterogeneous sensors (radar, electro-optical/infrared, radio-frequency sensing, acoustic, telemetry) and from a digital twin that exercises the same contracts.
2. Normalise those observations into a canonical schema with explicit coordinate reference system (CRS), time bases, quality metadata, and provenance.
3. Maintain tracks and cross-sensor associations under clutter, dropout, and clock skew.
4. Produce behaviour indicators and graded risk assessments that preserve uncertainty rather than collapsing it into false binaries.
5. Present an operator console that supports evidence inspection, alert prioritisation, incident workflow, and human decisions over *authorised response categories*.
6. Persist tamper-evident audit records for material events and decisions.
7. Integrate, if at all, only via category-level handoff to externally authorised systems — never via in-platform weapon or electronic-attack control loops.

The software-engineering challenge is to organise services, events, stores, and APIs so that these obligations remain enforceable under partial failure, schema evolution, model change, and organisational turnover over a multi-year ownership horizon.

1.2 Why software architecture dominates early delivery risk

In sensing-and-decision platforms, teams often over-index on detector accuracy or fusion mathematics while under-investing in the substrate that makes those components operable: durable streaming, idempotency, projection rebuild, contract versioning, and fail-visible degradation. Bass, Clements and Kazman argue that architecture is the primary carrier of quality attributes; algorithms alone do not confer availability, auditability, or evolvability [*Software Architecture in Practice*, 3rd/4th ed., Addison-Wesley]. Kleppmann similarly frames modern data systems as compositions of storage, encoding, and derived views under reliability constraints [*Designing Data-Intensive Applications*, O'Reilly, 2017]. For counter-UAS decision support, those constraints are sharpened by safety: a silent wrong track or an unlogged decision is worse than a delayed, explicitly degraded picture.

1.3 Scope, non-goals, and integrity constraints

In scope: software architecture styles; event streaming; API and schema contracts; consistency models for tracks and decisions; observability; testing and chaos; an implementation blueprint matching Counter-Swarm services (normaliser, tracking, fusion, risk, operator-api, and adjacent services); trade-offs; failure modes; build sequencing.

Out of scope: weapon aimpoints, fuse/arm logic, jammer waveforms, kinetic guidance, proprietary effector SDKs as core dependencies, and any code path that could autonomously cause harmful physical effects.

Citation integrity: only real books, peer-reviewed or workshop papers with established bibliographic identity, standards, and primary technical documentation are cited. No fabricated DOIs.

1.4 Research questions

ID	Question
RQ1	Which architectural style best satisfies latency, evolvability, safety isolation, and operability for multi-sensor defensive C2 software?
RQ2	How should Kafka-compatible event streaming be partitioned, versioned, and consumed to support tracking/fusion without inventing tracks under failure?
RQ3	What consistency and API patterns keep operator projections trustworthy while allowing asynchronous processing?
RQ4	How can correctness of safety-relevant software properties be <i>evidenced</i> (contracts, simulation, chaos) without claiming formal verification of the entire system?
RQ5	What incremental build order yields a minimum reliable system aligned to Counter-Swarm services?

1.5 Method and contribution

This monograph is a design-science synthesis: it derives architectural decisions from stated requirements and quality attributes, compares alternatives with explicit gains/losses, maps decisions onto Counter-Swarm artefacts already drafted in Stage 0, and proposes evaluation

instruments (contracts, golden simulation, chaos mapped to FMEA). Contributions are:

1. A quality-attribute-driven comparison of monolith vs modular services vs event-driven cores for counter-UAS decision support.
2. A service-and-event blueprint coherent with Counter-Swarm ICDs.
3. A correctness-and-operations programme that treats safety interlocks as testable software properties.
4. A build guide that prefers boring, proven infrastructure unless a documented justification exists.

1.6 Document map

Section 2 surveys related software-engineering and systems literature. Section 3 binds requirements and quality attributes. Section 4 compares architectural styles. Section 5 treats event streaming. Section 6 covers API design. Section 7 addresses consistency. Section 8 covers observability. Section 9 covers testing. Section 10 presents the Counter-Swarm implementation blueprint. Sections 11–12 analyse trade-offs and failure modes. Section 13 discusses proving correctness. Section 14 is a build guide, followed by deep dives on module boundaries (14A), performance (14B), security engineering (14C), service-specific concerns (14D–F), consistency examples (14G), observability patterns (14H), testing artefacts (14I), adjacent schools (14J), and ownership (14K). Section 15 concludes. References and appendices follow.

2. Related Work

2.1 Software architecture and quality attributes

Bass, Clements and Kazman’s *Software Architecture in Practice* provides the vocabulary of tactics for availability, modifiability, performance, security, and testability used throughout this monograph. ISO/IEC 25010 defines a product quality model (functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, portability) that is useful for structuring non-functional requirements even when numerical targets are site-specific [ISO/IEC 25010:2011 / successor revisions]. Parnas’s classic criteria for module decomposition remain relevant: hide design decisions likely to change [*On the Criteria To Be Used in Decomposing Systems into Modules*, Communications of the ACM, 1972]. Conway’s organisational observation — that system structure mirrors communication structure — warns against microservice proliferation that outruns team boundaries [Conway, *How Do Committees Invent?*, Datamation, 1968].

2.2 Microservices, modularity, and distributed data

Newman’s *Building Microservices* (O’Reilly; 1st ed. 2015, 2nd ed. 2021) is the primary practitioner reference for service boundaries, independent deployability, and the operational cost of fine-grained decomposition. Fowler’s public essay *Microservices* (martinfowler.com, with Lewis, 2014) and subsequent notes on microservice trade-offs caution that distribution is not free: network partitions, partial failure, and operational complexity accompany independent

scaling. Fowler's earlier *Patterns of Enterprise Application Architecture* (Addison-Wesley, 2002) remains useful for layering, mapping, and gateway patterns inside each service. Richardson's *Microservices Patterns* (Manning, 2018) catalogues saga, outbox, CQRS, and API composition patterns used in Section 7. Evans's *Domain-Driven Design* (Addison-Wesley, 2003) and Vernon's *Implementing Domain-Driven Design* (Addison-Wesley, 2013) inform bounded contexts: sensor ingest, tracking/fusion, risk/policy, and operator decision support should not share a single mutable model.

2.3 Event-driven integration and log-centric systems

Hohpe and Woolf's *Enterprise Integration Patterns* (Addison-Wesley, 2003) remains the canonical catalogue of messaging styles (pub/sub, competing consumers, idempotent receiver, dead-letter channel, claim check). Kreps, Narkhede and Rao introduced Kafka as a distributed commit-log messaging system for high-throughput log processing [*Kafka: a Distributed Messaging System for Log Processing*, NetDB workshop, 2011]. Kreps's subsequent writing on the log as a unifying abstraction for data integration influenced event-sourcing-adjacent designs in industry. Stopford's *Designing Event-Driven Systems* (O'Reilly, 2018) elaborates stream processing patterns on Kafka. The Apache Kafka documentation (current project docs under kafka.apache.org) is treated here as the normative technical reference for topics, partitions, consumer groups, idempotent producers, transactions, and retention — not as a research paper, but as the authoritative engineering baseline.

2.4 Data-intensive systems and consistency

Kleppmann's *Designing Data-Intensive Applications* synthesises replication, partitioning, transactions, batch/stream processing, and derived-data systems with unusual clarity for practitioners. Vogels's *Eventually Consistent* (Communications of the ACM, 2009) and Brewer's CAP conjecture and later clarifications [Brewer, *CAP Twelve Years Later*, Computer, 2012] frame what operator-facing projections may promise under partition. Lamport's *Time, Clocks, and the Ordering of Events in a Distributed System* (Communications of the ACM, 1978) underpins why sensor timestamps and platform receive times must both be carried. Gray and Reuter's *Transaction Processing* (Morgan Kaufmann) and Bernstein and Newcomer's *Principles of Transaction Processing* remain the background for when local ACID is appropriate (decision+audit+outbox) versus when stream processing must settle for at-least-once plus idempotency.

2.5 Reliability, operations, and chaos

Nygard's *Release It!* (Pragmatic Bookshelf; 2nd ed. 2018) catalogues stability patterns (circuit breakers, bulkheads, timeouts) applicable to inference gateways and integration clients. Google's *Site Reliability Engineering* (Beyer et al., O'Reilly, 2016) and *The Site Reliability Workbook* supply SLI/SLO vocabulary used in Counter-Swarm observability design. Principles of Chaos Engineering (principlesofchaos.org; industry consensus document associated with Netflix's pioneering work) motivate controlled experiments that falsify availability assumptions. Humble and Farley's *Continuous Delivery* (Addison-Wesley, 2010) and Forsgren, Humble and Kim's *Accelerate* (IT Revolution, 2018) ground the build guide's emphasis on automated regression and deployment safety.

2.6 Observability tooling lineage

OpenTelemetry project documentation defines vendor-neutral metrics, logs, and traces transport (OTLP) used in Counter-Swarm’s ICD-14. Prometheus documentation defines the pull-based metrics model common in Stage-1 lab deployments. These are engineering references, not theoretical contributions, but they constrain what “observable” means in implementable terms.

2.7 Human factors adjacent to software contracts

Endsley’s work on situation awareness [*Toward a Theory of Situation Awareness in Dynamic Systems*, Human Factors, 1995] motivates evidence-linked tracks and explicit missing-information fields in risk events. Nielsen’s usability heuristics [*10 Usability Heuristics for User Interface Design*, NN/g; and *Usability Engineering*, Morgan Kaufmann, 1993] are cited lightly where API/DTO design affects visibility of system status, error prevention, and recognition over recall — properties that belong partly to product design but impose requirements on software contracts (health banners, lag age, conflict flags). Parasuraman, Sheridan and Wickens on levels of automation [*A Model for Types and Levels of Human Interaction with Automation*, IEEE Trans. SMC-A, 2000] reinforce that recommendations must remain distinct from commands in event taxonomies.

2.8 Tracking, fusion, and sensing software (adjacent, non-exhaustive)

Classical multi-target tracking and fusion literature (e.g., Bar-Shalom, Li and Kirubarajan, *Estimation with Applications to Tracking and Navigation*; Blackman and Popoli, *Design and Analysis of Modern Tracking Systems*; Hall and Llinas on multisensor data fusion surveys) informs *what* tracking/fusion services must compute. This monograph does not re-derive filters; it specifies software boundaries, state ownership, and failure behaviour around those computations. ML evaluation metrics (MOTA/IDF1 and related tracking metrics) appear only as release gates referenced from Counter-Swarm testing strategy.

2.9 Security and safety standards as software constraints

NIST SP 800-207 (*Zero Trust Architecture*) and OWASP ASVS inform service identity, least privilege, and API hardening expectations. Functional safety standards such as IEC 61508 are not claimed as certification targets here; they are acknowledged as the broader industrial context for fail-safe design culture. Counter-Swarm’s hard rule — no autonomous destructive actuation in-platform — is a product safety boundary enforced by architecture and tests, not a claim of SIL certification.

2.10 Gap this monograph addresses

Public materials on Kafka, microservices, and SRE are abundant; public materials on *defensive counter-UAS decision-support software engineering* that refuse weaponisation while specifying contracts, projections, and chaos mapping are sparse. Industry C2 systems exist, but their

internal software architecture is rarely published at monograph depth. This document fills that niche for Counter-Swarm ownership: a defensible, citation-honest software-engineering argument tied to a concrete service catalogue.

3. Requirements and Quality Attributes

3.1 Stakeholders and outcomes

Primary users are security operators, sensor operators, and incident commanders. Secondary users are analysts (replay), administrators, governance officers, and integrators. Meaningful outcomes are: timely coherent air picture; graded risk with rationale and missing-info; human decisions with audit; graceful degradation under sensor or infrastructure failure; and zero ability for the platform itself to execute harmful effects.

3.2 Functional requirement clusters (software-relevant)

Drawn from Counter-Swarm docs/systems/requirements.md and condensed for architecture:

Cluster	Software implications
Sensor adapters	Pluggable processes; vendor types stop at adapter boundary
Normalisation	Stateless validators; quarantine topics; CRS/time transforms versioned
Event streaming	Durable bus; schema registry; consumer lag as SLO
Detection / tracking / fusion	Stateful processors; model version on detections; evidence links on tracks
Behaviour / risk	Rule+model hybrid; never emit <code>human.decision</code>
Alerts / approval	Policy version pins; RBAC on decision write
Operator console APIs	REST + realtime; DTO versioning
Audit / integration	Append-only audit; category-only external handoff; outbox
Simulation	Same contracts as live adapters

3.3 Quality attribute scenarios

Following Bass et al., quality attributes are expressed as scenarios:

Latency. A validated observation enters the bus; within NFR budgets (see Counter-Swarm NFR-LAT-*), a track update is projectable to the operator console; alert policy evaluation after risk update completes within its p95 budget.

Availability. Single-site MVP targets (e.g., 99.5% as drafted) mean planned degradation modes rather than pretending continuous perfection: banners for lag, coasting tracks with uncertainty growth, decision freeze if audit cannot write.

Safety. Under any combination of service crashes, the platform cannot invoke weapon interfaces; recommendation events cannot be misinterpreted by integration as effector commands; demo configuration cannot target production integration endpoints without governance-approved config.

Security. Service-to-service identity (mTLS/SASL), user OIDC, RBAC on decision and admin routes, input validation on all external surfaces, least privilege on object-store and bus ACLs — aligned with zero-trust orientation (NIST SP 800-207).

Explainability / auditability. Risk assessments carry rationale factors and evidence identifiers; material actions produce audit records; model versions appear on detections.

Modifiability. Adding a sensor modality requires a new adapter + schema fields (additive), not a rewrite of tracking. Changing alert policy is a versioned config publish, not a redeploy of fusion.

Testability. Every ICD entry has fixtures; simulation can inject delay/drop/dup/corrupt; chaos experiments map to FMEA IDs.

Operability. Health/readiness endpoints, RED metrics, consumer lag, quarantine rate, and runbooks exist before field trials.

3.4 Constraints and assumptions

Assumptions include simulation-first development, secured site network, synchronisable clocks with residual skew handling, single-site MVP, and external systems that accept abstract categories (Counter-Swarm A-01–A-06). If any assumption fails — for example, if clocks cannot be bounded — association quality and thus software SLOs for track coherence must be renegotiated; architecture should not silently invent certainty.

3.5 Quality model mapping (ISO/IEC 25010)

ISO/IEC 25010 characteristic	Counter-Swarm emphasis
Functional suitability	Evidence-backed tracks and graded risk
Performance efficiency	End-to-end latency budgets; backpressure
Compatibility	Canonical schema; Kafka API portability
Usability	Status visibility; conflict flags; Nielsen-aligned feedback
Reliability	Degradation modes D1–D6 in FMEA
Security	AuthZ, audit fail-closed on decisions
Maintainability	Modular services; contract tests
Portability	Compose lab → hardened staging → field

4. Architectural Styles Compared

4.1 Evaluation criteria

Styles are scored against: sensor-rate decoupling; independent scale of GPU detection vs CPU tracking vs API; blast-radius isolation; replay/auditability; operational burden for a small skilled team; safety boundary clarity; and time-to-minimum-reliable-system.

4.2 Style A — Modular monolith (“C2 application”)

Structure. One deployable with internal modules (ingest, track, risk, API) sharing a database.

Gains. Simple local reasoning; single transactional story; fewer network failure modes; fast early prototyping.

Losses. Sensor ingest load couples to API availability; ML runtimes hard to isolate; independent scaling poor; schema changes risk wide regressions; team parallelisation limited (Conway).

Failure modes. A memory leak in detection can take down decision capture. A blocking inference call can stall HTTP workers.

When it becomes a liability. As soon as live multi-sensor rates and model serving appear — typically before field utility.

Verdict for Counter-Swarm. Acceptable only as a short-lived spike to validate schemas; rejected as production target (architecture/system.md §9.A).

4.3 Style B — Fine-grained microservices everywhere

Structure. Many independently deployed services per noun (per filter stage, per widget backend, etc.).

Gains. Theoretical scale purity; technology diversity.

Losses. Newman repeatedly emphasises distributed-monolith risk: chatty sync calls, unclear ownership, high cognitive load for tracing, and premature fragmentation (*Building Microservices*). Fowler’s microservices essay stresses that microservices demand mature ops. Operational burden dominates early programme value.

Failure modes. Cascading timeouts; version skew across dozens of APIs; inconsistent authz.

Verdict. Reject extreme split; prefer domain-aligned modularity.

4.4 Style C — Event-driven modular services (selected)

Structure. Domain services own their write models; asynchronous events on a durable log couple producers and consumers; operator query paths use APIs over projections (CQRS-leaning without mandatory full event sourcing of every entity).

Gains. Natural fit to asynchronous sensors; replay for recovery and after-action; decoupling of normaliser throughput from risk logic; clear audit subsample points; simulation injects at the same bus boundary.

Losses. Eventual consistency between bus and UI projections; harder end-to-end reasoning; need for idempotent consumers and schema discipline (Kleppmann; Hohpe & Woolf).

Why selected. Matches Counter-Swarm architectural drivers: late/missing data is normal; safety boundary can be enforced at event-type and integration seams; Stage 0 ADRs already propose Kafka-API bus and modular services.

4.5 Hybrid note: modular monolith *inside* services

Each service may be a small modular monolith internally (clean modules, one deployable). Newman's guidance is to start with larger services and split when scaling or team boundaries demand it. Counter-Swarm's catalogue (normaliser, tracking, fusion, risk, operator-api, ...) is already near the right granularity: split by change rate and statefulness, not by every function.

4.6 Comparison table

Criterion	Monolith	Fine microservices	Event-driven modular (selected)
Sensor decoupling	Weak	Possible but sync-chatty	Strong via bus
Replay	Hard	Ad hoc	Native if log retained
Ops burden (MVP)	Low	Very high	Medium
Safety isolation	Weak	Variable	Strong if event types enforced
Consistency simplicity	High (local ACID)	Low	Medium (explicit)
Fit to Counter-Swarm	Spike only	No	Yes

4.7 Architectural decision record (software)

ADR	Decision	Alternatives rejected	Primary sources
SE-ADR-001	Event-driven modular services	Monolith target; nanoservices	Newman; Fowler; Kleppmann
SE-ADR-002	Kafka-compatible durable log	Cloud-only pub/sub as sole bus; ad hoc queues	Kreps et al.; Kafka docs; Stopford
SE-ADR-003	CQRS-leaning read models for UI	Single shared DB for all reads/writes	Richardson; Fowler POEAA
SE-ADR-004	Outbox for integration handoff	Dual-write bus+DB without outbox	Richardson; Kleppmann
SE-ADR-005	Category-only integration events	Effector-shaped payloads	Safety requirements

4.8 Architecture tactics applied (Bass et al.)

Quality	Tactics used in Counter-Swarm SE design
Availability	Replication (bus/DB), degradation modes, health checks, restart
Performance	Partition parallelism, claim-check large payloads, coalesce WS
Modifiability	Adapters, schema versioning, policy packs, inference gateway
Security	mTLS/SASL, RBAC, least-privilege ACLs, input validation
Testability	ICD fixtures, sim adapters, chaos hooks
Auditability	Append-only decision/audit, model versions on detections

Tactics are chosen deliberately; adding a tactic (e.g., multi-region routing) without a scenario is complexity theatre.

4.9 Interface-first delivery as a process tactic

Counter-Swarm’s ICD rule — no implementation against undocumented interfaces — is a process tactic that reduces integration defects. It echoes Parnas’s information hiding: the interface is the stable face; internals may churn.

4.10 Why “boring” is a research-justified preference

Programme constitution prefers proven infrastructure. That preference is not anti-intellectual: Kleppmann and Newman both document costs of novel distributed behaviour. Research contribution here is *selection under constraint*, not invention of a new broker.

5. Event Streaming (Kafka / Kreps Lineage)

5.1 Why a commit log

Kreps, Narkhede and Rao positioned Kafka as a distributed messaging system built around a persistent log partitioned for throughput and consumer parallelism [NetDB 2011]. For counter-UAS ingest, the log provides: (1) durable buffering when consumers lag; (2) ordered partitions for per-sensor sequencing; (3) multiple independent consumer groups (tracking, quality, audit sampler, analytics export); (4) retention-based replay for projection rebuild after failure — a property Kleppmann highlights when discussing event logs as sources of truth for derived data.

5.2 Kafka-compatible semantics required by Counter-Swarm

Normative behaviours (Apache Kafka documentation):

- **Topics** for event families: `observation.v1`, `detection.v1`, `track.update.v1`, `behaviour.indicator.v1`, `risk.assessment.v1`, `alert.v1`, quarantine and DLQ topics.

- **Partitions** keyed by `sensor_id` or `site_id` as appropriate to preserve per-key ordering.
- **Consumer groups** for scalable competing consumers on stateless stages (normaliser output consumers that are sharded); sticky or single-writer patterns for stateful tracking/fusion shards.
- **Idempotent producers** to reduce duplicate publishes on retry.
- **ACLs** per service principal.
- **Retention** sized for recovery/replay windows (Counter-Swarm draft: 7–30 days environment-specific), not infinite storage of raw video (that belongs in object storage with claim-check references — Hohpe & Woolf claim check pattern).

Redpanda is an acceptable Kafka-API implementation for lab/MVP ops simplicity (architecture/tradeoffs.md); the software contracts must remain Kafka-API portable.

5.3 Event taxonomy and ownership

Event	Owner (producer)	Critical consumers	Ordering key
<code>observation.v1</code>	normaliser	detection, tracking, quality	<code>sensor_id</code>
<code>observation.quarantine.v1</code>	normaliser	DE/SG tooling	<code>sensor_id</code>
<code>detection.v1</code>	detection	tracking	<code>sensor_id / frame_id</code>
<code>track.update.v1</code>	tracking, fusion	behaviour, risk, realtime	<code>site_id + track_id</code>
<code>behaviour.indicator.v1</code>	behaviour	risk, UI	<code>site_id</code>
<code>risk.assessment.v1</code>	risk	alert, UI, audit	<code>site_id + track_id</code>
<code>alert.v1</code>	alert	UI, notify	<code>site_id</code>
<code>recommendation.v1</code>	decision support	UI, approval	<code>site_id</code>
<code>human.decision.v1</code>	approval	audit, outbox	<code>site_id</code>
<code>integration.handoff.v1</code>	integration	external, audit	<code>site_id</code>
<code>sensor.health.v1</code>	adapters	UI, risk context	<code>sensor_id</code>

Non-negotiable: `recommendation.v1` is not `human.decision.v1`. Integration must not accept effector-like fields (ICD-11).

5.4 Schema registry and evolution

Use a schema registry with **BACKWARD** compatibility for value schemas (Counter-Swarm ICD-02). Consumers must ignore unknown fields (tolerant reader pattern; Kleppmann’s discussion of schema evolution). Major versions appear in event type names (`observation.v2`) when semantics break. Contract tests gate merges.

Encoding choices (Avro vs JSON Schema vs Protobuf) are secondary to compatibility discipline; team familiarity and registry support dominate. Document the choice in the tech matrix; do not dual-encode without need.

5.5 Processing semantics

At-least-once delivery is the default practical model. Exactly-once *end-to-end* across heterogeneous side effects (DB projection + outbound HTTP) is rarely free; Kafka transactions help within the Kafka ecosystem but do not replace idempotent handlers and transactional outbox for decisions (Kleppmann; Richardson).

Handler rule: every consumer side effect must be idempotent under duplicate `observation_id / detection_id / track_update_id / decision_id`.

5.6 Watermarks, late data, and clock skew

Fusion and tracking must accept out-of-order events within a bounded watermark; beyond watermark, route to late-processing or mark quality LATE rather than silently corrupting state (architecture/data-flow.md §8–9). Carry both `timestamp_utc` and `received_at_utc`. Skew beyond threshold sets `CLOCK_SKEW` quality flags — Lamport’s lesson applied to sensor reality: without careful time, association is fiction.

5.7 Backpressure and load shedding

When consumers lag, prefer explicit degradation (UI lag banner, shedding non-critical analytics consumers) over dropping canonical observations without metrics. Adapters use bounded local buffers and shed-oldest with metrics (ICD-01). Normaliser never “fixes” invalid data by guessing geodesy.

5.8 Stream processing vs service consumers

Kafka Streams / Flink-style jobs can implement windowed analytics later. MVP preference: ordinary consumer services with clear ownership, because debugging fusion state machines in a general stream DSL often reduces operability for mixed teams. Revisit when analytics throughput demands it (Stopford’s patterns remain available).

5.9 Consumer patterns in detail

Competing consumers (stateless). Multiple normaliser-output processors in one group share partitions. Suitable for detection fan-out when detection is horizontally scalable. Commit offsets only after durable side effects succeed, or accept idempotent replay (Kafka docs; Kleppmann).

Single-writer stateful consumers. Tracking shard owns a partition set. Rebalance must pause writes and hand over state or rebuild. Prefer static membership or careful assignors in hardened stages.

Multicast via multiple groups. Tracking, quality, and audit samplers each use distinct group IDs to receive the same `observation.v1` stream — the core fan-out advantage of the log relative to point-to-point queues (Kreps et al.).

Request-reply anti-pattern. Avoid implementing operator queries as “publish request, wait for reply topic” when REST over projections suffices. Mixing styles increases timeout debt (Nygard).

5.10 Poison messages and DLQ policy

A repeatedly failing record must not block a partition forever. Policy: after N attempts, publish to DLQ with error metadata; advance offset; page if DLQ rate exceeds budget (FMEA FM-06). Schema validation upstream reduces poison rates; fuzz tests keep them low.

5.11 Compacted topics for latest state (optional)

Kafka log compaction can store latest sensor health or latest risk per key. Use carefully: compaction is not a general database. Hot Postgres remains system of record for operator queries in Counter-Swarm MVP.

5.12 Multi-site future (explicit non-MVP)

Active-active multi-region event ordering is a research and ops problem beyond MVP (architecture/system.md rejects multi-region active-active for MVP). Design site_id into keys now so future federation does not rewrite history.

6. API Design

6.1 Dual-stack rationale

Counter-Swarm selects REST+OpenAPI for operator and integration surfaces; gRPC for high-rate adapter→normaliser and inference where profiling justifies it; WebSocket/SSE for realtime UI (architecture/tradeoffs.md). Fielding’s REST dissertation [*Architectural Styles and the Design of Network-based Software Architectures*, UC Irvine, 2000] justifies resource-oriented operator APIs; it does not require REST for internal sensor pipes.

6.2 Operator API principles

operator-api responsibilities:

- Authenticated CRUD-ish access to incidents, alerts (ack), configuration views, evidence queries, and decision submission endpoints that delegate to approval.
- DTO mapping from hot-store projections — not leaking internal event schemas wholesale to browsers.
- Versioned under /api/v1; breaking changes require /api/v2.
- Idempotency keys on decision POST.
- Hard deny when audit path cannot confirm write (fail-closed).

Realtime gateway fans out track/alert projections; sticky sessions optional. On disconnect, UI reconnects with backoff (ICD-09). Nielsen's "visibility of system status" heuristic implies the API must expose lag age and degradation flags, not only happy-path track JSON.

6.3 Integration API principles

External handoff payloads contain: `decision_id`, `category`, `site_id`, `track_ids`, `evidence_digest`, `timestamp`. Forbidden: `aimpoints`, RF jam parameters, arming flags. mTLS + allowlists. Async retry with DLQ; local decision remains valid if handoff fails (ICD-11).

6.4 Internal service APIs

Prefer events for cross-domain state propagation. Synchronous calls are reserved for: inference gateway, identity token validation, and tight adapter→normaliser links on edge. Chatty sync webs of microservices recreate the distributed monolith Newman warns against.

6.5 AuthN/AuthZ

- Users: OIDC bearer access tokens + refresh; short-lived access (Counter-Swarm security notes align with common practice of hours-scale access with refresh — exact TTLs are policy).
- Services: mTLS or SASL to bus; scoped tokens to object store.
- Authorisation: RBAC permissions such as `decision.write`; server-side checks only — never trust UI hiding of buttons.
- AI assistant tools: allowlisted Operator API tools; assistant cannot write `human.decision.v1` (ICD-13).

6.6 Error model

Return generic messages to operators; log detail server-side. Distinguish 401/403/409/422/429/503. Rate-limit decision and login-adjacent routes. Validate all redirect/webhook-like inputs if ever introduced; SSRF guards on integration URLs (software architecture notes).

6.7 Pagination, filtering, and evidence assembly

Evidence queries should return identifiers and digests first, with claim-check fetch of raw media from object storage. This keeps API payloads bounded and aligns with Hohpe & Woolf's claim check pattern.

7. Data Consistency

7.1 Consistency domains

Domain	Model	Mechanism
Canonical observations on bus	Durable append; per-partition order	Kafka log
Track state (tracking/fusion)	Single-writer per shard; rebuildable	Consumer + snapshot store
Hot DB projections	Eventually consistent with bus	Idempotent projectors
Human decisions + audit	Strong local consistency	DB transaction + outbox
Integration handoff	At-least-once with status	Outbox poller + DLQ
UI session cache	Ephemeral	Redis; never source of truth

7.2 Single-writer principle

Each entity class has one writable owner (software architecture notes). Tracks are not updated by risk. Risk does not mutate track kinematics. Approval does not rewrite risk history; it appends decisions.

7.3 CQRS-leaning without dogma

Operator reads come from projections optimised for map/list/timeline queries. This is CQRS as Richardson describes it — separation of write and read models — not a mandate to event-source every row. Snapshots in Postgres+PostGIS hold current tracks, incidents, and alert state; the bus retains the recent event history for rebuild.

7.4 Outbox pattern for decisions

Dual-writing to DB and bus without a transactional outbox creates silent divergence under crash (widely documented; see Richardson *Microservices Patterns* and Kleppmann's discussion of derived data). Counter-Swarm: commit `human.decision + audit + outbox` row in one DB transaction; publisher relays to bus/integration.

7.5 Conflicts and duplicates

Condition	Consistency handling
Duplicate idempotency key	Dedup; metric
Class conflict	Preserve distribution; <code>CLASS_CONFLICT</code>
Spatial incompatibility	Reject association; separate tracks
Over-merge vs under-merge	Prefer conservative association; operator-visible flags
Stale projection	Version/timestamp on DTO; UI can show staleness

7.6 CAP in operational terms

Under partition between edge and central, prefer buffering and degraded coverage over inventing a confident global track picture (FMEA degradation modes). Brewer's CAP framing is a reminder, not a slogan: for operator safety, **consistency of uncertainty signalling** beats availability of false precision.

7.7 Geospatial consistency

CRS transforms are versioned library code (geo-sdk). Incorrect transforms are high-severity (FMEA FM-07). Unit tests with known control points are mandatory; "looks right on map" is not a consistency proof.

8. Observability

8.1 Telemetry model

Per Counter-Swarm observability architecture and Google SRE practice:

Signal	Use
Metrics (OTLP/Prometheus)	RED for APIs; consumer lag; quarantine rate; infer latency
Structured logs	correlation_id across ingest→UI
Traces (OTLP)	critical path spans: adapt→normalise→track→risk→alert→UI
Health	/healthz, /readyz distinct (liveness vs dependency readiness)

8.2 SLIs and example SLOs

SLI	Draft SLO intent
API availability	Match site availability target
Consumer lag	Alert before UI staleness intolerable
Decision persist p95	Interactive budget (~300 ms persist as ICD-10 draft)
Audit write success	Near-perfect; page on failure
Quarantine rate	Baseline + anomaly detect

Exact numbers live in requirements (NFR-LAT-, *NFR-OBS*-); architecture must make them measurable.

8.3 Required dashboards

SYSTEM HEALTH, MODEL HEALTH, SENSOR HEALTH, DATA QUALITY, SECURITY, OPERATOR ACTIVITY — as specified in docs/systems/observability.md. Operator-activity metrics (ack time, decisions/hour) close the loop between software performance and human

factors without claiming to replace dedicated UX studies.

8.4 Tracing the critical path

Propagate context from adapter request IDs into event headers (where size permits) and into projection logs. OpenTelemetry documentation describes context propagation conventions; implement them early — bolting on traces after fusion bugs appear is expensive.

8.5 Alerting for platform ops vs C2 alerts

Keep namespaces separate: pages for bus down / audit fail are operations; `alert.v1` tiers To–T3 are mission. Confusing them creates alert fatigue (product concern) and noisy on-call (SRE concern).

9. Testing Strategy

9.1 Layered tests (Counter-Swarm aligned)

Layer	Targets
Unit	Geo transforms, association gates, risk rules, authz helpers, validators
Integration	adapter→normaliser, normaliser→bus, tracking←detections, approval→audit, handoff→mock
Contract	OpenAPI; schema BACKWARD; ICD fixtures
Simulation	Scenario pack; golden runs; scorer gates for tracking/fusion
Chaos	Mapped to FMEA IDs
Security	AuthZ, IDOR, injection, rate limits, schema fuzz, assistant prompt-injection corpus
Safety	Recommendation cannot handoff; effector fields rejected; demo cannot hit prod URL; audit fail blocks decision
Human factors	Offline scripted trials (non-CI) feeding product design

9.2 Contract testing as architecture enforcement

Consumer-driven contracts (e.g., Pact-style tooling, or schema registry compatibility checks plus golden JSON fixtures) prevent “works in my compose stack” drift. Every ICD ID needs a fixture before producer schema merge (`architecture/interfaces.md` §5).

9.3 Simulation as first-class harness

Digital twin adapters publish through the same contracts as real adapters (`docs/systems/digital-twin.md`). This is not optional polish: it is how software engineering validates tracking/fusion under controlled faults (delay/drop/dup/corrupt) before hardware.

9.4 Performance and soak

Soak tests should watch for consumer leak, projection lag growth, and DB bloat under sustained synthetic rates. Nygard's stability patterns apply: timeouts on inference, bulkheads between IO pools.

9.5 What testing cannot claim

Tests do not prove absolute absence of bugs. They provide evidence against stated hazards. Formal methods may apply later to small kernels (e.g., authz policy evaluation); they are not assumed for the full distributed system in MVP.

9.6 Contract test example narrative

Producer normaliser proposes adding optional `quality.calibration_age_s`. Contract suite: (1) registry BACKWARD check passes; (2) golden observation still consumed by tracking fixture; (3) UI DTO ignores unknown nested fields; (4) documentation row added to ICD-02. Merge blocked if tracking fixture used a strict decoder that rejects unknowns — fixing the consumer is part of the change.

9.7 Chaos experiment template

1. **Title / FMEA ID**
2. **Hypothesis** (e.g., "Under broker kill, UI shows D3 within 30s and decisions remain fail-closed if audit impacted")
3. **Blast radius** (lab only)
4. **Observability watched**
5. **Steady state**
6. **Inject**
7. **Observe**
8. **Stop / rollback**
9. **Learning filed** (ADR or runbook update)

This follows Principles of Chaos Engineering structure without theatrical production outages.

9.8 Safety regression suite as release blocker

Any PR touching approval, integration-api, event enums for decisions/recommendations, or authz libraries must run the safety suite. Broader nightly runs cover full graph. This is how software engineering encodes the safety architecture continuously, not as a one-time review slide.

9.9 Mutation testing (selective)

For authz helpers and risk rule evaluators, mutation testing can increase confidence that unit tests actually catch faults. Apply selectively; whole-repo mutation is often cost-ineffective early.

10. Implementation Blueprint Matching Counter-Swarm Services

10.1 End-to-end software pipeline

```
sensor-adapter-* / sim-adapter
  → normaliser
  → event-bus (observation.v1)
  → detection (optional path) → detection.v1
  → tracking → track.update.v1
  → fusion → fused track.update.v1 + evidence links
  → behaviour → behaviour.indicator.v1
  → risk → risk.assessment.v1
  → alert → alert.v1
  → operator-api + realtime-gateway → console
  → approval → human.decision.v1 (+ audit + outbox)
  → integration-api → category handoff / mock
```

10.2 normaliser

Responsibility. Validate candidate observations; CRS/time normalisation; enrich with quality; enforce idempotency; publish `observation.v1` or `observation.quarantine.v1`.

State. Stateless; horizontal pod autoscaling friendly.

Inputs. ICD-01 candidate observations (gRPC/queue/HTTPS push).

Outputs. ICD-02 Kafka produces.

Key libraries. Schema validation; geo-sdk transforms; metrics for quarantine rate and skew.

Failure behaviour. Quarantine on invalid; never crash the bus; never invent coordinates.

Tests. Property tests on CRS; fuzz malformed payloads; duplicate key dedup.

10.3 tracking

Responsibility. Multi-object tracking / state estimation from detections and/or observations; emit `track.update.v1` with covariance and evidence identifiers.

State. Stateful per shard; careful failover; snapshots to hot DB.

Failure behaviour. Coast with increasing uncertainty; mark COASTING; do not fabricate detections.

Tests. Unit gates; sim scenarios for clutter; MOTA/IDF1-style gates as agreed with ML/evaluation owners.

10.4 fusion

Responsibility. Cross-sensor association; fused tracks; evidence links; conservative merge policy.

State. Stateful; site-scoped.

Failure behaviour. Prefer under-merge over over-merge when uncertain (FMEA FM-11 vs FM-12); flag conflicts for UI.

Tests. Multi-modality sim scenarios (including RF-silent / modality missing); association rejection cases.

10.5 risk

Responsibility. Graded risk from tracks, behaviour, context; emit missing-information signals; version policy/model.

State. Mostly stateless compute with config/policy pins; projections for latest assessment per track.

Rules. Must not emit human decisions; must not call integration; LLM assistants must not own this path alone (architecture/system.md §10).

Tests. Policy golden files; monotonicity sanity checks where applicable; feature-flag rollback drill.

10.6 operator-api

Responsibility. REST DTOs for console; evidence queries; config reads; gateway to approval; health aggregation endpoints as needed.

State. Stateless; relies on hot DB + auth.

Companion. realtime-gateway for WS/SSE fanout.

Failure behaviour. Read-only cache banner if projections stale; block decisions if approval/audit down.

Tests. OpenAPI contract; IDOR tests on track/incident IDs; authz matrix.

10.7 Adjacent services (brief)

Service	Software note
detection	Model version on every event; fail-open/closed per route policy
behaviour	Indicators only; not decisions
alert	Policy id on every alert; storm aggregation
approval	RBAC; idempotent decision; transactional outbox
audit	Tamper-evident append; fail-closed coupling to decisions
inference-gateway	Timeouts; canary; rollback
integration-api	Category only; DLQ
sim-engine	Lab gated; fault injectors
ai-assistant	Tool allowlist; citations; off critical path

10.8 Data stores

- **Hot:** PostgreSQL + PostGIS (tracks, incidents, config, outbox).
- **Ephemeral:** Redis (sessions, rate limits).
- **Objects:** S3-compatible (raw media, models) with references from events.
- **Analytics:** export later; do not block MVP on warehouse.

10.9 Deployment topology (software view)

Central-primary hybrid: adapters at edge; fusion/risk/operator-api central (architecture/system.md §7). Compose for lab; harden to multi-broker / multi-AZ as programme stages demand (architecture/deployment.md). Raspberry Pi-class devices are not fusion brains.

10.10 Language and framework defaults

Python/FastAPI for many control-plane services; Go considered for ultra-high-rate adapters if profiling requires; React+TypeScript console (architecture/tradeoffs.md). These are defaults, not religion — justify deviations with measurements.

11. Trade-offs

11.1 Event-driven core vs sync request chains

Gains: decoupling, replay, fanout. **Losses:** mental model cost, eventual consistency. **Hard later if wrong:** bolting a bus onto a chatty monolith after field deployment. **Risk:** teams bypass bus with hidden sync calls — mitigate with ICD review and tracing.

11.2 Kafka-compatible vs simpler queues

Gains: ecosystem, retention, consumer groups. **Losses:** ops weight. **Mitigation:** Redpanda or managed Kafka for MVP. **Risk:** under-skilled ops — budget runbooks and chaos early.

11.3 Conservative fusion vs aggressive merge

Gains: fewer catastrophic over-merges. **Losses:** fragmented tracks, operator clutter. **Product interaction:** UI must show fragments honestly. **Risk:** tuning pressure from demos to “look clean” — resist with sim-scored gates.

11.4 Dual REST/gRPC

Gains: right tool per boundary. **Losses:** two stacks. **Accept** for clarity; avoid a third (GraphQL) until a concrete operator query pain appears.

11.5 Exactly-once aspirations vs idempotent reality

Chasing global exactly-once often delays delivery. Prefer idempotent consumers + outbox. Document residual duplicate windows.

11.6 Edge autonomy vs central coherence

More edge processing reduces bandwidth and improves local resilience; increases fleet ops cost and version skew. MVP central-primary is a deliberate trade for ownership simplicity.

11.7 Assistant on vs off critical path

Assistants improve summarisation but hallucinate (FMEA FM-25). Keep off critical path; require citations; audit tool calls.

11.8 Summary matrix

Decision	Gain	Loss	Residual risk
Event-driven modular	Decoupling, replay	Complexity	Hidden sync bypass
Kafka API	Proven streaming	Ops	Mispartitioning
Projections for UI	Fast reads	Lag/staleness	Silent staleness without banners
Outbox decisions	Integrity	Extra moving part	Publisher bugs
Category-only integration	Safety	Less “wow” automation	Social pressure to widen API

12. Failure Modes

12.1 Principles (from Counter-Swarm FMEA)

Fail visible; degrade coverage rather than invent tracks; block external handoff under uncertainty; preserve audit; recommendations never become commands on failure.

12.2 Software-centred failure catalogue (selected)

ID	Mode	Software detection	Software mitigation
FM-05	Bus outage	Broker health	Backpressure; banner; multi-broker
FM-06	Poison message	Error budget / DLQ	Skip-to-DLQ; schema guards
FM-07	Bad CRS	Unit tests; spot checks	Versioned transforms; kill-switch adapter
FM-08	Clock skew	Skew metrics	Quality flags; widen gates
FM-09	Hot DB down	Ready checks	Failover; freeze decisions if needed
FM-10	Divergent tracks	Track KPIs	Replay rebuild
FM-11/12	Over/under merge	Sim + operator reports	Conservative thresholds; UI flags
FM-13/14	Model down/bad	Latency/errors; calibration	Passthrough policy; canary disable
FM-15	Risk rule bug	Policy tests	Version pin
FM-16	Alert storm	Alert rate SLI	Aggregate/suppress
FM-18/20	Approval/audit fail	Health; write errors	Fail-closed decisions
FM-23	Duplicate burst	Idempotency	Dedup keys
FM-25	Assistant hallucination	Grounding checks	Citations; optional disable

12.3 Degradation modes as software states

Implement explicit mode flags consumed by operator-api and UI:

- D1 Coverage reduced
- D2 Model degraded
- D3 Stale picture (lag age)
- D4 Decision freeze
- D5 Integration offline
- D6 Assistant off

Modes must be first-class in DTOs, not tribal knowledge.

12.4 Recovery playbook (software)

1. Restore bus/DB health.
2. Replay observations from retention watermark.
3. Rebuild projections.
4. Verify audit continuity.

5. Exit degraded mode with operator acknowledgement.

Nygard-style runbooks should name the dashboards and the replay job, not merely “restart pods.”

13. Proving Correctness: Contracts, Simulation, and Chaos

13.1 What “correct” means here

For this platform, correctness is a bundle of properties:

1. **Interface conformance** — producers/consumers honour ICD schemas and auth.
2. **Idempotent processing** — duplicates do not corrupt tracks or double-apply decisions.
3. **Safety interlocks** — no path from recommendation to effector-shaped integration without human decision; forbidden fields rejected.
4. **Degradation honesty** — lag and conflicts surface to operators.
5. **Audit completeness** for material decisions under fail-closed rules.
6. **Regression bounds** on tracking/fusion via simulation scorers.

This is not a claim of formal verification of the entire distributed system.

13.2 Contracts

- Schema registry compatibility gates in CI.
- OpenAPI provider/consumer checks for operator-api and integration-api.
- Golden event fixtures per ICD ID.
- Architectural fitness functions (e.g., ArchUnit-like or custom lint) forbidding integration clients inside risk or tracking services.

13.3 Property-based and metamorphic tests

Where feasible: CRS round-trip properties; association symmetry expectations; risk policy monotonicity on selected features. Metamorphic testing (known in SE literature for ML/systems) can check that delaying all timestamps by a constant does not change spatial association beyond tolerance.

13.4 Chaos engineering

Follow Principles of Chaos Engineering: hypothesis → experiment → learn. Map experiments to FMEA IDs:

Experiment	Hypothesis	FMEA
Kill bus broker	UI shows stale banner; no silent invent	FM-05
Inject corrupt message	Lands in quarantine/DLQ; consumers continue	FM-06/22
Duplicate observations	Track counts stable	FM-23
Clock skew injection	Quality flags; no crash	FM-08
Pause audit store	Decisions blocked	FM-20
Inference timeout	Policy path engages; badge visible	FM-13

Chaos without observability is vandalism; require dashboards first.

13.5 Safety proofs as automated tests

Property	Test oracle
Recommendation cannot call integration	Deny observed
Effector-like JSON fields	Schema reject
Demo flag → prod URL	Config blocker
Audit write fail	Decision commit aborts

These tests are the software-engineering backbone of the safety architecture diagram in `architecture/system.md §8`.

13.6 Evidence pack for release

A release candidate should attach: contract CI green; sim scenarios threshold pass; chaos drill notes for critical FM-IDs; SBOM/CVE gate; migration/rollback notes. Forsgren et al.'s *Accelerate* associates rigorous automated testing and delivery discipline with performance — here it is also a safety argument.

14. Build Guide

14.1 Principles

Interfaces before components; simulation before hardware; boring infrastructure first; no effector drivers; author Victor.I ownership continuity via ADRs.

14.2 Stage-aligned software delivery

Aligned to Counter-Swarm roadmap spirit:

Stage 0 — Contracts and architecture (current). Freeze conceptual schemas; ICD index; FMEA draft; compose skeleton empty of unsafe integrations.

Stage 1 — Vertical thin slice. sim-adapter → normaliser → bus → trivial projector → operator-api hello track list; observability baseline (Prometheus/Grafana or equivalent); schema registry.

Stage 2 — Tracking baseline. Deterministic or classical tracker; hot DB projections; realtime gateway; lag banners.

Stage 3 — Risk rules + alerts + approval. Policy versioning; audit; outbox; safety tests green.

Stage 4 — Fusion across sensors. Conservative association; evidence UI; sim gates.

Stage 5 — Hardening. Chaos suite; multi-broker; authz pen-tests; performance soak.

Stage 6+ — ML routes, assistant (bounded), HIL. Canary inference; assistant off-critical-path; hardware-in-loop adapters under authorisation.

14.3 Repository and module layout (recommended)

```
/schemas          # event and DTO contracts
/services
  /normaliser
  /tracking
  /fusion
  /risk
  /operator-api
  /approval
  /integration-api
  /realtime-gateway
  /sensor-adapter-sim
/libs
  /geo-sdk
  /authz
  /observability
/tests
  /contracts
  /simulation
  /chaos
/architecture      # ADRs already present
/docs
```

Enforce dependency direction: adapters depend on schemas; core services do not depend on console; integration-api does not depend on ML.

14.4 Local developer path

1. Run Compose: bus, schema registry, Postgres, Redis, MinIO, observability.
2. Produce sim observations.
3. See quarantine metrics if schemas break.
4. Run contract tests before push.
5. Never point demo config at real external systems.

14.5 CI gates (minimum)

Unit + contract + sim smoke on PR; nightly full scenarios; container/dep scan; disallow merge on BACKWARD incompatibility; safety test suite required on paths touching approval/integration.

14.6 Configuration and secrets

All secrets via environment/secret manager; .env gitignored; integration allowlists in audited config; policy packs versioned and referenced from alerts/decisions.

14.7 Rollback

- Config/policy rollback by version pin.
- Model rollback via inference-gateway.
- Service rollback via standard deployment.
- Data: projection rebuild from bus retention; decisions immutable (compensating actions are new audited events, not silent edits).

14.8 Definition of done for “minimum reliable system”

- Thin slice live in lab with degradation banners.
- Human decision + audit fail-closed verified.
- Category handoff to mock only.
- Chaos FM-05 and FM-20 drilled once.
- No weapon/effector code paths in repo.

14A. Module Boundaries, Dependency Rules, and Evolutionary Design

14A.1 Bounded contexts for Counter-Swarm

Evans’s domain-driven design advises aligning module boundaries with linguistic and change boundaries. Counter-Swarm’s software contexts are:

Context	Language	Ownership
Ingest & quality	observation, quarantine, skew, CRS	normaliser, adapters, quality
Kinematic picture	track, association, coast, covariance	tracking, fusion
Assessment	behaviour indicator, graded risk, missing info	behaviour, risk, alert
Human authority	recommendation, decision, category, audit	approval, audit, integration
Operator experience	DTO, viewport, incident, ack	operator-api, realtime-gateway, console
Model serving	model_id, score, canary	inference-gateway, detection/classification
Twin	scenario, fault injector, scorer	sim-engine, sim-adapters

Anti-corruption layers sit at adapter boundaries: vendor SDKs never leak into fusion. Vernon's emphasis on context mapping applies when integrating an external SOC tool: translate categories, do not share databases.

14A.2 Allowed dependency graph

```
adapters → schemas, geo-sdk, obs-lib
normaliser → schemas, geo-sdk
tracking/fusion → schemas, geo-sdk (read), hot-db client
risk → schemas, policy packs (read)
operator-api → hot-db, authz, DTO schemas
approval → hot-db, authz, audit client, outbox
integration-api → outbox reader, external clients (allowlisted)
console → operator-api, realtime only
```

Forbidden edges: risk → integration-api; tracking → approval; console → bus (browsers must not consume Kafka directly in production designs); ai-assistant → decision write.

Fitness functions in CI should fail the build when import graphs violate these edges. This is architecture as code, not as slideshow.

14A.3 Hexagonal ports inside each service

Cockburn's hexagonal architecture (ports and adapters) remains a practical internal style: domain logic depends on ports; Kafka, Postgres, and HTTP adapters implement them. Fowler's POEAA gateway and mapper patterns fit the same idea. For *risk*, policy evaluation is the domain; Kafka consumer and Postgres projector are adapters. This keeps unit tests free of brokers.

14A.4 Evolutionary pressure points

Expect change in: sensor modalities, alert policies, UI layouts, model routes, and site CRS conventions. Hide those behind adapters, policy packs, DTO versioning, and inference-gateway respectively (Parnas). Do not hide human-decision semantics behind "flexible JSON" —

controlled vocabularies exist for safety.

14B. Performance Engineering Without Premature Optimisation

14B.1 Measure the critical path first

Until sim load tests exist, optimisation is speculation. Instrument:

1. Adapter transform time
2. Normaliser validate+produce time
3. Tracking update latency end-to-end
4. Projection lag
5. API and WS fanout latency

Google SRE's guidance on SLIs applies: percentile latency matters more than averages for operator trust.

14B.2 Partition and key design impacts throughput

Too few partitions limit parallelism; too many increase overhead and rebalance pain (Kafka documentation). Key by `sensor_id` for observation order; re-key carefully when moving to track-centric topics — re-keying is a stream-processing stage with its own failure modes (Stopford). Hot keys (one very busy radar) may need local sharding strategies documented in an ADR.

14B.3 Stateful service scaling

Tracking/fusion cannot naively HPA like normaliser. Options: partition assignors that pin track shards; standby replicas; rebuild-from-log on fail. Document RPO/RTO for track state. Prefer explicit “not ready” during rebuild over serving empty confident pictures.

14B.4 Payload discipline

Large EO frames do not belong on the bus. Store objects; pass URIs and digests (claim check). This protects broker memory and consumer lag. Kleppmann's discussion of data encoding and derived data reinforces keeping the log manageable.

14B.5 Language performance policy

Python is acceptable for control services at Counter-Swarm rates if scaled out and if inference is isolated. If profiling shows adapter CPU saturation, consider Go for that adapter only — Newman's advice against speculative rewrites applies. Premature “rewrite everything in X” is a programme risk.

14C. Security Software Engineering (Defensive Platform)

14C.1 Threat-informed design hooks

From Counter-Swarm threat-model themes and OWASP ASVS: validate all ingest; authenticate every service hop; authorise every decision; prevent IDOR on track/incident identifiers; rate-limit; sanitise assistant tools; scan dependencies. NIST SP 800-207 zero-trust orientation means no implicit trust of “inside the VLAN.”

14C.2 Bus as a security boundary

Kafka ACLs are not optional decoration. Producers should only write their topics; consumers only read what they need. Quarantine topics may have narrower read sets (DE/SG). Audit subsample consumers should not be writable by UI roles.

14C.3 Decision path hardening

- CSRF protections as applicable to cookie-based sessions (if used); prefer bearer tokens for SPA.
- Idempotency keys prevent double-submit under retry.
- Server-side RBAC for `decision.write`.
- Immutable audit entries; admin “edit history” is a new compensating event, never a silent UPDATE of the original.

14C.4 Supply chain

SBOM generation, container scanning, pinned base images, and signed artefacts belong in CI before field stages. Humble & Farley’s continuous delivery pipeline is incomplete without security gates when the system influences physical-world response categories.

14C.5 Explicit non-goals (security)

This monograph does not provide offensive cyber tradecraft. Hardening is defensive: reduce attack surface of the decision-support platform itself.

14D. Deep Dive: normaliser Implementation Concerns

14D.1 Validation pipeline stages

1. **Structural parse** — reject non-JSON/Avro; metric `parse_fail`.
2. **Schema validate** — required fields, enums, ranges; quarantine with reason code.
3. **Idempotency check** — redis/db bloom or KV with TTL keyed by `idempotency_key`.
4. **Time normalisation** — parse to UTC; record `received_at_utc`; flag skew vs reference clock.
5. **CRS transform** — versioned geo-sdk; never silent no-op on unknown CRS.

6. **Quality enrich** — pass through SNR/FOV/calibration age when present.

7. **Produce** — idempotent producer to `observation.v1`.

14D.2 Quarantine as a product feature

Quarantine is not a trash bin. It is how data engineering and sensor ops learn about broken adapters. Expose rates on DATA QUALITY dashboards; alert on spikes; retain samples under policy. Hohpe & Woolf's dead-letter channel pattern is the ancestor.

14D.3 Exactly what not to put in normaliser

No tracking, no risk scoring, no UI DTOs, no model inference. Normaliser's job is to make a trustworthy canonical event. Overloading it recreates a monolith at the edge of the bus.

14E. Deep Dive: tracking and fusion Software Concerns

14E.1 State machine obligations

Regardless of filter mathematics (Kalman, MHT, etc.), software must:

- Apply association gates with logged parameters (versioned).
- Emit track updates with evidence observation IDs.
- Handle duplicates without double-counting.
- Coast with covariance growth and COASTING quality.
- Survive restart via snapshot + bus replay.

Bar-Shalom et al. and Blackman & Popoli define estimation quality; software engineering defines operability of that estimation under failure.

14E.2 Fusion evidence links

Every fused track should point to contributing modality observations/detections. This enables Endsley-aligned situation awareness in the UI and supports after-action review. Without evidence links, "trust the track" becomes social pressure rather than engineering.

14E.3 Over-merge as a software-policy choice

Association thresholds are configuration under change control. Demo environments that loosen thresholds for prettier maps must not silently promote to production. Policy version IDs on track updates (or on fusion config referenced by updates) make regressions auditable.

14E.4 Interaction with detection failures

When inference is down, ICD-04 allows modality-native passthrough detections per policy. Tracking must tolerate gaps: absence of ML detections is not permission to invent them. FMEA FM-13/14 bind here.

14F. Deep Dive: risk and operator-api

14F.1 Risk as decision support, not authority

`risk.assessment.v1` carries graded risk, rationale factors, missing-information codes, and versions. It does not open gates. Alert policy consumes risk; humans consume alerts and evidence. Parasuraman et al.'s automation levels remind designers that moving from “recommend” to “act” is a deliberate organisational decision — and Counter-Swarm forbids in-platform harmful act.

14F.2 Missing-information as a first-class field

Operators should see *why* confidence is limited (no RF, stale EO, high skew). Software that omits missing-info fields forces operators to guess — a Nielsen error-prevention failure and a safety smell.

14F.3 operator-api read models

Suggested resources (illustrative, not final OpenAPI):

- GET `/api/v1/tracks` — filters, bbox, stale flag
- GET `/api/v1/tracks/{id}` — kinematics, evidence, risk summary
- GET `/api/v1/alerts` — tier, ack state
- POST `/api/v1/alerts/{id}/ack`
- GET `/api/v1/incidents` / POST
- POST `/api/v1/decisions` — body with category + idempotency key
- GET `/api/v1/system/status` — degradation modes D1–D6, lag age

Fielding's uniform interface discipline: use HTTP semantics honestly; do not overload GET with side effects.

14F.4 Realtime fanout

Realtime gateway subscribes to projection changes or track/alert topics (service-side) and pushes to sessions. Backpressure: drop or coalesce high-rate track ticks per client rather than unbounded queue memory growth (Nygard stability). Coalescing must not hide degradation banners.

14G. Consistency Worked Examples

14G.1 Duplicate observation

Radar adapter retries; same `idempotency_key` arrives twice. Normaliser dedups; tracking sees one logical observation. Metrics increment `dedup_hits`. Without this, track quality metrics lie.

14G.2 Decision under audit outage

Operator submits HEIGHTEN_MONITORING. Approval begins transaction; audit write fails. Transaction aborts; API returns 503 with generic message; no outbox row; no handoff. Operator sees failure — not silent success (ICD-10; FMEA FM-20).

14G.3 Integration offline after successful decision

Decision commits; outbox filled; external system down. Retry/DLQ; handoff status=failed; local decision remains valid (ICD-11). UI shows handoff failure without erasing the decision. This is deliberate consistency: authority record vs notification delivery.

14G.4 Projection lag during bus recovery

Tracks in UI are 45s stale. system/status reports D3 with lag age. Map dims or banners per product rules. Software correctness here is *honesty*, not low lag alone.

14G.5 Split-brain fusion shards (anti-pattern)

Two fusion instances both write the same track_id without partition discipline. Divergent state appears (FM-10). Correctness requires single-writer assignment — a software deployment concern as much as an algorithm concern.

14H. Observability Implementation Patterns

14H.1 Correlation identifiers

Generate correlation_id at adapter ingress; propagate via event headers and HTTP; log structured fields. When an operator asks “why did risk spike?”, engineers need join keys across services — OpenTelemetry spans help, but logs must still be greppable under incident pressure.

14H.2 RED and USE

For request services: Rate, Errors, Duration (SRE community practice popularised alongside Google SRE materials). For resources: utilisation, saturation, errors (USE method as taught in widely used performance analysis practice; apply pragmatically to CPU, broker disk, consumer threads).

14H.3 Cardinality control

Do not label Prometheus metrics by raw track_id (cardinality explosion). Aggregate; use exemplars/traces for instances. Kafka consumer lag by partition is acceptable; lag by every observation ID is not.

14H.4 Synthetic checks

Black-box: inject a labelled sim observation periodically; expect it to appear in a projection within SLO. This catches “all green / nothing flows” failures that unit tests miss.

14I. Testing Artefacts Catalogue

Artefact	Owner	Gate
ICD golden JSON/Avro	DE + SE	PR
OpenAPI spec	SE	PR
Schema compatibility	DE	PR
Sim scenario YAML	SE + Sim	Nightly
Scorer thresholds	ML/DS + SE	Release
Chaos experiment runbooks	SE + SRE	Stage gate
Safety interlock tests	SG + SE	PR on related paths
AuthZ matrix tests	SG + SE	PR
Load/soak profiles	SE	Pre-field

14I.1 Flaky test policy

Flakes destroy trust in gates (*Accelerate* culture). Quarantine flakes with owners; never ignore safety-test flakes.

14I.2 Test data ethics and realism

Sim scenarios should include benign clutter, birds/balloons analogues if relevant to site, RF silence, and swarm-like multiplicity — without requiring real harmful payloads in engineering environments. Fault injectors cover delay/drop/dup/corrupt as first-class (ICD-15).

14J. Comparison to Adjacent Architectural Schools

14J.1 Pure event sourcing of all entities

Pros: complete audit of state transitions. Cons: complexity of rehydration, snapshotting, and GDPR-like deletion constraints if personal data appears. Counter-Swarm uses an event log for sensor/decision flows and snapshots for operational entities — a pragmatic hybrid Kleppmann would recognise as derived-data thinking rather than dogma.

14J.2 Actor models (e.g., Akka-style)

Actors can model tracks well. Operational familiarity and ecosystem (Kafka+Postgres) currently beat actor runtime novelty for this programme unless a team already owns that stack. Justify with ADR if chosen.

14J.3 Serverless functions per event

Fine for bursty low-state transforms; poor for low-latency stateful fusion without heavy external state. Cold starts and observability fragmentation are real costs. Not selected for MVP core path.

14J.4 Data mesh rhetoric

Domain-oriented data ownership rhymes with modular services, but full mesh organisational overhead is unjustified for single-site MVP. Keep canonical observation as the stability hinge without standing up multiple data products prematurely.

14K. Long-Horizon Ownership Concerns

14K.1 Documentation that survives authors

README and ADRs assume a new reader and gone author (programme constitution). This monograph is part of that memory, but runnable contracts and dashboards matter more than prose.

14K.2 Skill mix

Teams need: distributed systems basics, Postgres, Kafka ops literacy, web security, and enough estimation literacy to not misuse track covariances. Hiring only “full-stack CRUD” engineers without streaming experience is a latent failure mode.

14K.3 Technical debt ledger

Explicit debts: eventual projection lag; dual REST/gRPC; Python performance headroom; single-site only. Debts become defects when denied.

14K.4 Three-year regret test

Would we regret Kafka-compatible bus? Unlikely if schemas stay clean. Would we regret nanoservices? Yes. Would we regret putting effector APIs “just for demo”? Catastrophically — do not.

15. Conclusion

15.1 Answers to research questions

RQ1. An event-driven modular service architecture best matches multi-sensor defensive decision-support quality attributes, provided granularity follows domain boundaries (Newman) and teams avoid nanoservice sprawl (Fowler's cautions).

RQ2. Kafka-compatible streaming with per-sensor or per-site partitioning, schema registry BACKWARD evolution, idempotent consumers, and explicit late-data handling provides the substrate for tracking/fusion without requiring exact global synchrony (Kreps et al.; Kafka docs; Kleppmann).

RQ3. Separate consistency domains: strong local transactions for decisions/audit/outbox; eventual projections for UI; conservative fusion; visible staleness (Richardson; Brewer/Vogels context).

RQ4. Correctness is evidenced through ICD contracts, simulation golden runs, chaos mapped to FMEA, and automated safety interlocks — not by unverifiable slogans.

RQ5. Build a simulation-backed vertical slice first, then stateful tracking, then risk/approval/audit, then fusion hardening — matching Counter-Swarm staging.

15.2 Pre-ship gate (software engineering)

Question	Answer
Failure modes understood?	Yes at draft FMEA depth; site Sev calibration pending
Observable in prod?	Yes if OTLP/RED/lag shipped before field
Safe rollback?	Yes via config/model/replay design
Complexity proportional to value?	Yes if nanoservices resisted
Want 3-year ownership?	Yes if contracts and ADRs remain authoritative

15.3 Residual risks

Social pressure to widen integration APIs; demo-driven aggressive fusion tuning; hidden sync coupling; under-investment in chaos; assistant misuse as authority. These are ownership risks as much as technical ones.

15.4 Final stance

The software-engineering task for Counter-Swarm is to make the defensive air picture and graded risk support **reliable, auditable, and fail-visible**, while making harmful actuation **architecturally unreachable**. Event-driven modular services on a Kafka-compatible log, with disciplined APIs and a ruthless testing programme, are the boring, defensible means to that end.

16. Evaluation Framework and Metrics for Software Releases

16.1 Distinguishing algorithm metrics from software metrics

Tracking MOTA/IDF1 and detector mAP are necessary release inputs from ML/evaluation owners, but they do not measure whether the platform is operable. Software releases need a parallel scorecard:

Software metric	Intent
Contract suite pass rate	Interface integrity
Mean time to detect bus lag (synthetic)	Observability efficacy
Decision fail-closed under audit kill	Safety interlock
Projection rebuild time from watermark	Recoverability
Quarantine false-positive rate	Adapter quality vs schema rigidity
p95 ingest→projection latency	Operator freshness
Change fail rate / lead time	Delivery health (<i>Accelerate</i>)

16.2 Acceptance for Stage promotion

Stage $N \rightarrow N+1$ should require written evidence: which SLIs were met in lab; which FMEA drills ran; which debts remain. Vague “demo worked” is not acceptance.

16.3 Human-in-the-loop evaluation interface

Software must expose timing hooks for product/human-factors trials (time-to-evidence, ack time) without coupling console rendering to server clocks incorrectly — use server event timestamps in the timeline API.

17. Migration, Schema Evolution, and Compatibility Playbooks

17.1 Additive field rollout

1. Merge schema with optional field (BACKWARD).
2. Deploy consumers that tolerate the field.
3. Deploy producers that populate the field.
4. Only then make UI depend on it.

Reversing steps 2–3 causes production poison. Kleppmann’s schema evolution guidance is operationalised here as ordered deploys.

17.2 Breaking event major versions

Publish `track.update.v2` alongside `v1` during dual-run; migrate consumers; retire `v1` after lag metrics show zero. Dual-run costs disk and CPU — budget retention.

17.3 Hot DB migrations

Expand-contract pattern: add columns nullable → backfill → switch readers → enforce constraints → remove old. Never lock operator tables without a maintenance window plan. Decisions table is append-mostly; prefer insert-only designs.

17.4 Replay after poison schema

If a bad producer ships, stop producer; quarantine topic may hold garbage; fix schema; replay from last known good offset into rebuilt projections. Practice this in lab — untested replay is superstition.

18. Operator-API Error and Degradation Contract (Normative Intent)

Condition	HTTP / WS behaviour	DTO fields
Unauthenticated	401	—
Forbidden	403	—
Unknown track	404	—
Validation error	422	field errors
Rate limited	429	retry-after
Audit/IAM freeze	503 on decision writes	degradation: D4
Stale projections	200 with data	degradation: D3, lag_age_ms
Integration offline	200 on decision	handoff_status: failed/pending

Nielsen's visibility heuristic is satisfied only if the console *and* API agree on degradation enums. Divergent enums are defects.

19. Worked Build: First Vertical Slice Tasks

Concrete engineering backlog for Stage 1 (no effector work):

1. Define `observation.v1` JSON Schema/Avro in `/schemas` with examples.
2. Stand up Compose: Kafka-compatible bus, registry, Postgres, Redis, MinIO, Prometheus, Grafana.

3. Implement sensor-adapter-sim emitting candidates with deliberate fault modes.
4. Implement normaliser with quarantine topic and metrics.
5. Implement minimal projector writing tracks table (even placeholder kinematics).
6. Implement operator-api GET /tracks + GET /system/status.
7. Add OpenAPI and one consumer contract test.
8. Add synthetic canary observation job.
9. Document runbook: “bus down”, “quarantine spike”.
10. Safety placeholder: integration-api exists only as mock with deny-by-default routes.

Exit: a new engineer can run Compose, see a track, break schema, see quarantine rise, and restore.

20. Threats to Validity and Honesty Limits

20.1 External validity

Site CONOPS, sensor suites, and legal constraints vary. Architectural style recommendations may need retuning for ultra-denied networks or air-gapped certification regimes not specified here.

20.2 Construct validity of “correctness”

Contracts and chaos evidence correctness of stated properties only. They do not prove absence of unknown hazards.

20.3 Citation and depth limits

This is Pass-1 monograph depth per docs/research/README.md: research-structured, landmark-cited, implementation-bound. It is not a multi-year dissertation with empirical field trials. Pass-2 may deepen proofs and surveys without inventing citations.

20.4 Adversarial sensing

Software can quarantine nonsense and flag conflicts; it cannot by itself defeat a sophisticated physical spoof without sensing and ML contributions. Claims otherwise would be dishonest.

References

1. Bar-Shalom, Y., Li, X. R., & Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation*. Wiley.
2. Bass, L., Clements, P., & Kazman, R. *Software Architecture in Practice* (3rd ed. 2012 / 4th ed. 2021). Addison-Wesley.

3. Bernstein, P. A., & Newcomer, E. (2009). *Principles of Transaction Processing* (2nd ed.). Morgan Kaufmann.
4. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering*. O'Reilly.
5. Blackman, S., & Popoli, R. (1999). *Design and Analysis of Modern Tracking Systems*. Artech House.
6. Brewer, E. (2012). CAP Twelve Years Later: How the “Rules” Have Changed. *Computer*, 45(2), 23–29.
7. Conway, M. E. (1968). How Do Committees Invent? *Datamation*, 14(4), 28–31.
8. Endsley, M. R. (1995). Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors*, 37(1), 32–64.
9. Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley.
10. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
11. Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate*. IT Revolution.
12. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
13. Fowler, M., & Lewis, J. (2014). Microservices.
<https://martinfowler.com/articles/microservices.html>
14. Gray, J., & Reuter, A. (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
15. Hall, D. L., & Llinas, J. (1997). An Introduction to Multisensor Data Fusion. *Proceedings of the IEEE*, 85(1), 6–23.
16. Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns*. Addison-Wesley.
17. Humble, J., & Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
18. ISO/IEC 25010:2011. *Systems and software engineering — System and software Quality Models*. ISO.
19. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
20. Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: a Distributed Messaging System for Log Processing. *NetDB*.
21. Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
22. Newman, S. (2015; 2nd ed. 2021). *Building Microservices*. O'Reilly.
23. Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.
24. Nielsen, J. 10 Usability Heuristics for User Interface Design. Nielsen Norman Group.
<https://www.nngroup.com/articles/ten-usability-heuristics/>
25. NIST SP 800-207. (2020). *Zero Trust Architecture*. National Institute of Standards and Technology.
26. Nygard, M. T. (2018). *Release It!* (2nd ed.). Pragmatic Bookshelf.
27. OpenTelemetry Authors. OpenTelemetry Documentation. <https://opentelemetry.io/docs/>
28. OWASP. *Application Security Verification Standard (ASVS)*. <https://owasp.org/www-project-application-security-verification-standard/>
29. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, 30(3), 286–297.
30. Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12), 1053–1058.

31. Principles of Chaos Engineering. <https://principlesofchaos.org/>
32. Prometheus Authors. Prometheus Documentation. <https://prometheus.io/docs/>
33. Richardson, C. (2018). *Microservices Patterns*. Manning.
34. Stopford, B. (2018). *Designing Event-Driven Systems*. O'Reilly.
35. The Apache Software Foundation. Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
36. Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
37. Vogels, W. (2009). Eventually Consistent. *Communications of the ACM*, 52(1), 40–44.
38. Cockburn, A. Hexagonal architecture writings (ports-and-adapters pattern).
39. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
40. IEC 61508 series. *Functional safety of electrical/electronic/programmable electronic safety-related systems*. International Electrotechnical Commission. (Contextual; no SIL claim here.)
41. OpenAPI Initiative. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>
42. PostgreSQL Global Development Group. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
43. PostGIS Project. PostGIS Documentation. <https://postgis.net/documentation/>
44. Redis Ltd. / Redis documentation. <https://redis.io/docs/>
45. OWASP Top 10. <https://owasp.org/www-project-top-ten/>

Counter-Swarm internal artefacts (normative for implementation)

1. Counter-Swarm Defence. `architecture/system.md`, `architecture/data-flow.md`, `architecture/interfaces.md`, `architecture/failure-modes.md`, `architecture/tradeoffs.md`, `architecture/deployment.md`. Author: Victor.I.
2. Counter-Swarm Defence. `docs/systems/requirements.md`, `docs/systems/testing-strategy.md`, `docs/systems/observability.md`, `docs/software/architecture-notes.md`. Author: Victor.I.

Appendix A — Event Field Checklist for Engineers

When adding a field to `observation.v1` or `track.update.v1`:

1. Is it additive within major version?
2. Does any consumer break if absent?
3. Is PII/sensitive data minimised?
4. Is object payload claimed-checked rather than inlined?
5. Are metrics/logs updated?
6. Is an ICD fixture updated in the same PR?
7. Does UI need a degradation or conflict affordance?

Appendix B — Safety Interlock Checklist

1. No service role can publish effector-shaped integration payloads.
2. `recommendation.v1` $\neq$ `human.decision.v1`.
3. Integration schema deny-list tested in CI.
4. Demo/prod endpoint separation tested.
5. Audit failure blocks decision commit.
6. Assistant cannot write decisions.
7. Threat model reviewed when new egress added.

Appendix C — Glossary

Term	Meaning
Canonical observation	Schema-normalised sensor measurement event
Projection	Read model derived from events
Outbox	Transactional table for reliable publish
Quarantine	Topic/store for invalid observations
Category handoff	Abstract authorised response type to external system
Coasting	Track prediction without fresh associations
ICD	Interface Control Document entry

Appendix D — Partition Key Decision Table

Topic family	Recommended key	Rationale	Watch-outs
<code>observation.*</code>	<code>sensor_id</code>	Preserve per-sensor order	Hot sensor skew
<code>detection.*</code>	<code>sensor_id</code>	Align with obs	—
<code>track.update.*</code>	<code>site_id:track_id</code>	Single-writer fusion/tracking	Re-key from sensor stream
<code>risk.assessment.*</code>	<code>site_id:track_id</code>	Colocate with track consumers	—
<code>alert.*</code>	<code>site_id</code>	Site-wide policy	—
<code>human.decision.*</code>	<code>site_id</code>	Audit locality	Low volume
<code>sensor.health.*</code>	<code>sensor_id</code>	Latest health per sensor	Compaction optional

Appendix E — Idempotency Key Guidelines

Message	Idempotency basis
Candidate observation	Adapter-provided key or hash(sensor_id, timestamp, measurement digest)
detection.v1	detection_id UUID from producer
track.update.v1	(track_id, update_seq) or event_id
human.decision.v1	Client idempotency key header
integration.handoff.v1	decision_id (one active handoff attempt chain with status)

Hash-based keys must be stable across retries; include only fields that define the measurement identity.

Appendix F — Minimal Runbook Skeletons

F1 Bus unavailable

Symptoms: produce errors; lag unknown; UI D3/D5.

Actions: check brokers; freeze noncritical consumers; verify disk; restore; replay if needed; operator ack exit D3.

Verify: synthetic canary passes.

F2 Quarantine spike

Symptoms: DATA QUALITY alert.

Actions: sample quarantine reasons; roll back adapter if bad deploy; loosen only with ADR.

Verify: rate returns to baseline; no silent drop without metric.

F3 Audit write failure

Symptoms: decisions 503; SECURITY/SYSTEM pages.

Actions: do not bypass fail-closed; restore storage; confirm no partial outbox; communicate decision freeze to operators.

Verify: safety test still green; sample decision succeeds.

Appendix G — Mapping Quality Attributes to Counter-Swarm NFRs

QA scenario (Bass style)	Example NFR family
Ingest freshness	NFR-LAT-*
Trace across path	NFR-OBS-*
AuthZ on decisions	Security requirements / SG docs
Sim parity	FR-SEN-003 and twin docs
No autonomous harm	Safety MoSCoW won't-haves

Exact numeric NFRs live in docs/systems/requirements.md and may change with customer CONOPS; architecture must remain measurable against them.

End of monograph

Author: Victor.I